TASK 3 – Define the Constants

Bloom's Level: K3 – Apply
Marks: 3

Define the tariff values and limits as constants so that a tariff revision is made in one place only.

Discuss: • The use of #define, const variables and enum for different kinds of constants • The difference between a #define constant and a const variable (type checking, scope, debugging)

• Naming conventions for constants

Complete the Code Complete TODO 1 using only the macros defined at the top, and execute the program.

```
#include <stdio.h>

#define SLAB1_UNITS 100 /* first 100 units */

 #define SLAB2_UNITS 200 /* units 101 to 200 */

#define RATE1_PAISE 0 /* first slab is free */

#define RATE2_PAISE 225 /* Rs 2.25 per unit */

#define RATE3_PAISE 450 /* Rs 4.50 per unit above 200 */

 #define FIXED_CHARGE_PAISE 5000L /* Rs 50 fixed charge */

 /* Returns the bill in paise, so that no rounding error can occur. */ long compute_bill(int units)

{

    long paise = FIXED_CHARGE_PAISE;

   // TODO 1: Add the energy charge slab by slab using the macros: units above

   // SLAB2_UNITS at RATE3_PAISE, units from SLAB1_UNITS+1 to SLAB2_UNITS at

  // RATE2_PAISE, and the rest at RATE1_PAISE

  return paise;

}

 int main(void) {

       int tests[] = {0, 100, 150, 200, 350};

      int n = sizeof tests / sizeof tests[0];
```

```
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
       }
      return 0;
}
```

Expected output:

```
0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Change RATE2_PAISE to 230 and check that every bill changes consistently.

Alternative Concept and Complexity Analysis

If the tariff constants were written as const variables or as an enum instead of #define, what would change in the program? Compare type checking, visibility in the debugger and the memory used (a #define occupies no storage; a const variable may), and state whether the running time is affected.

Evidence Expected A single, well-named set of constants with a justified choice between #define, const and enum.

1. Use of #define, const and enum

#define: It is a preprocessor directive used to define symbolic constants. It replaces a macro name with its value before compilation. It is suitable for fixed tariff rates and billing limits.

Example:

```
#define RATE2_PAISE 225
```

const: It declares a variable whose value cannot be modified through that variable after initialization. It has a data type and follows normal C scope rules.

Example:

const int maximum_units = 200;

enum: It defines named integer constants. It is useful for a fixed set of categories or states.

Example:

enum Category { DOMESTIC, COMMERCIAL, INDUSTRIAL };

For this billing program, #define is appropriate because the question specifically requires using the macros defined at the top.

2. Difference between #define and const

Feature	**#define**	**const**
Processing	Preprocessor	Compiler
Data type	No inherent data type	Has a specified data type
Type checking	No type checking for the macro itself	Supports normal type checking
Scope	From definition until undefined or the end of the file	Follows C scope rules
Debugging	May be less convenient to inspect	Often easier to inspect
Memory	The macro itself occupies no storage	A variable may occupy storage
Running time	Usually no runtime cost for the replacement	Usually no additional runtime cost for a simple constant

3. Naming conventions for constants

Constants should have meaningful and descriptive names. Uppercase letters with underscores between words are a common naming convention.

Examples:

- SLAB1_UNITS
- SLAB2_UNITS

- RATE1_PAISE
- RATE2_PAISE
- RATE3_PAISE
- FIXED_CHARGE_PAISE

These names clearly identify the purpose of each constant and make tariff revisions easier.

4. Complete the Code

The TODO section is completed by calculating the energy charge slab by slab using the predefined macros.

The program first adds the charge for units above SLAB2_UNITS, then calculates the charge for units in the second slab. The first slab is free, so it contributes zero energy charge. The fixed charge is added to every bill.

5. Expected output with the original tariff

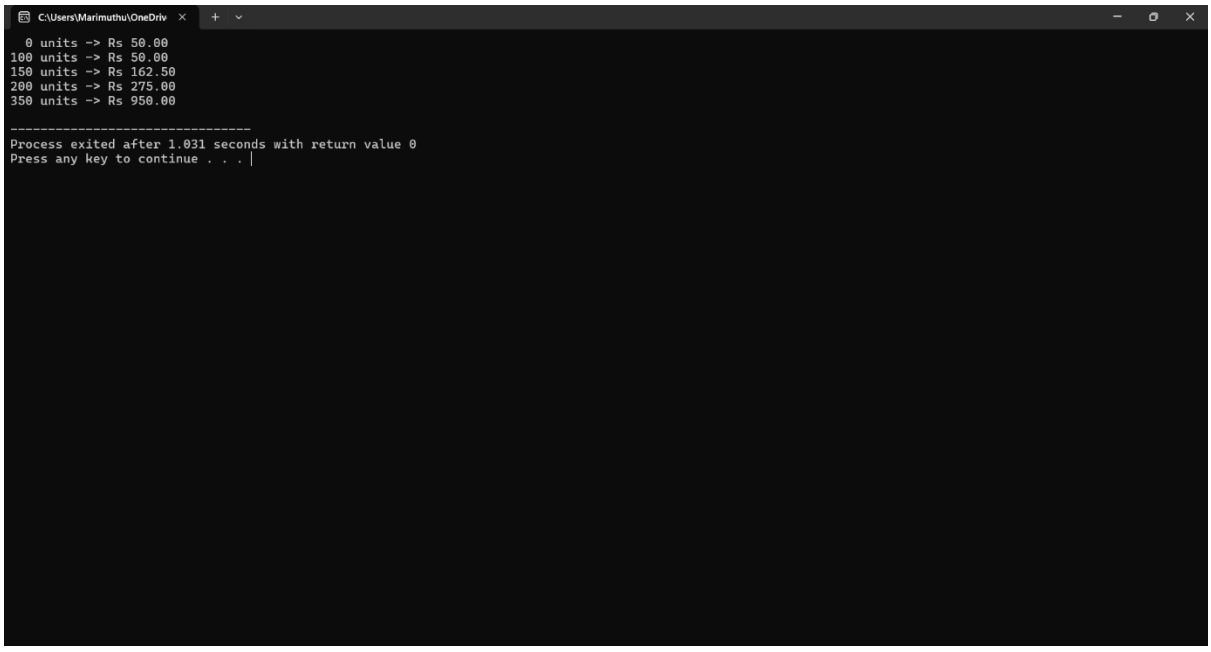
0 units -> Rs 50.00

100 units -> Rs 50.00

150 units -> Rs 162.50

200 units -> Rs 275.00

350 units -> Rs 950.00



6. Change RATE2_PAISE to 230

When RATE2_PAISE is changed from 225 to 230, the expected output is:

0 units -> Rs 50.00

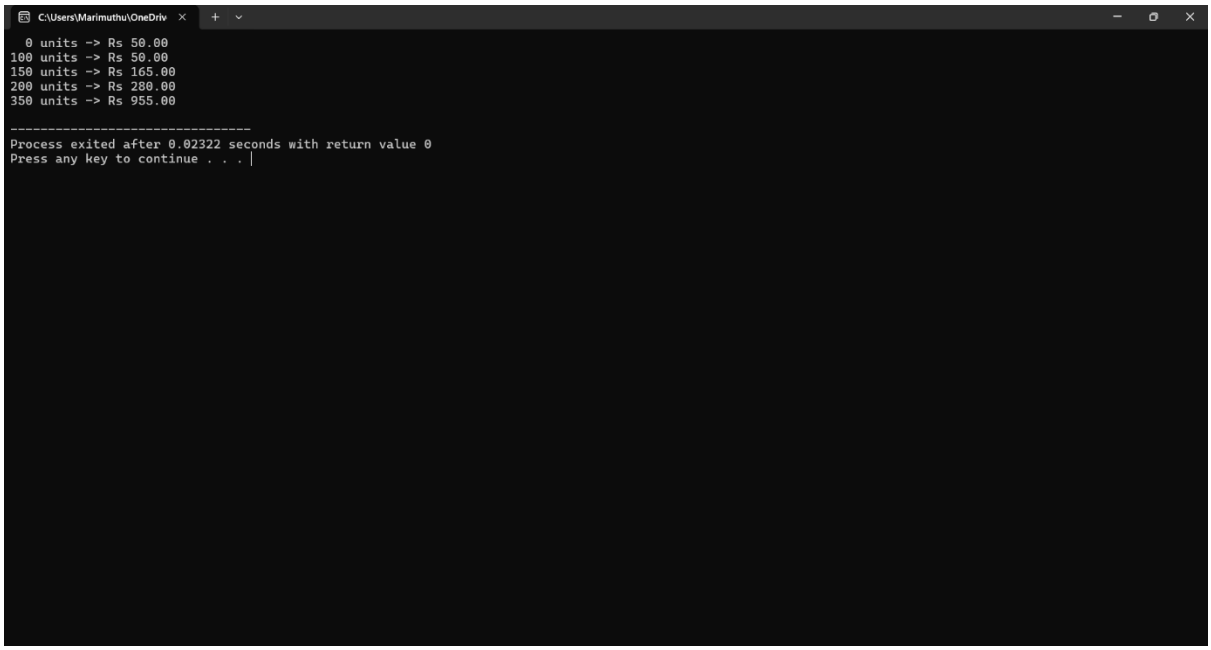
100 units -> Rs 50.00

150 units -> Rs 165.00

200 units -> Rs 285.00

350 units -> Rs 960.00

The bills for 150, 200 and 350 units increase because these bills include units charged at the second tariff rate. Bills for 0 and 100 units remain unchanged because the first slab is free.



7. Alternative Concept and Complexity Analysis

If const variables were used instead of #define, the tariff values would be declared as typed variables, such as const int RATE2_PAISE = 225;. This provides type checking and follows normal variable scope rules.

If enum were used, it would be suitable for integer-valued categories or states, but it would be less suitable for representing different tariff rates and billing limits.

Factor	**#define**	**const variable**	**enum**
Type checking	No inherent type	Yes	Integer constants

Debugging	May be less convenient	Often easier	Depends on compiler and debugger
Memory	No storage for the macro itself	May occupy storage	Enum constants do not occupy storage individually
Running time	(O(1)) per slab calculation	(O(1)) per slab calculation	(O(1)) for constant use

The program checks a fixed number of slabs regardless of the number of units consumed. Therefore, the time complexity is (O(1)), and the auxiliary space complexity is (O(1)).

8. Conclusion

Using well-named constants makes the billing program easier to maintain and reduces errors when tariff values change. The #define macros meet the requirements of this task. Storing charges in integer paise avoids floating-point rounding errors, and the slab-by-slab approach calculates the bill efficiently.